TEST PLAN & TEST CASES

GitHub REST API

API Test Suite Specification

Document version	1.0
Status	Draft
Date	13 October 2026
System under test	GitHub REST API (https://api.github.com), API version 2022-11-28
Reference	https://docs.github.com/en/rest
Test framework	Java 17, REST Assured 5.5.6, TestNG 7.11, Maven
Scope	13 user stories, 96 test cases

Table of Contents

Test Coverage Summary . . . 2
0. Conventions . . . 3
1. Authentication & Rate Limit . . . 3
2. Users . . . 4
3. Repositories . . . 4
4. Issues . . . 5
5. Repository content, branches & commits . . . 6
6. Search . . . 7
7. Gists . . . 8
8. Cross-cutting (contract) tests . . . 8
9. Suggested execution order & coverage . . . 8
11. Recommended Automation Approach . . . 8

Test Coverage Summary

Area	Test cases
1. Authentication & Rate Limit	8
2. Users	7
3. Repositories	19
4. Issues	28
5. Repository content, branches & commits	15
6. Search	7
7. Gists	6
8. Cross-cutting (contract) tests	6
Total	**96**

Priority	Meaning	Cases
P0	Smoke – must pass on every run	21
P1	Core regression	50
P2	Edge cases / nice to have	25

0. Conventions

- **Auth:** personal access token from env var `GITHUB_TOKEN` (never commit it). Header: `Authorization: Bearer <token>`.
- **Required scopes (classic PAT):** `repo`, `gist`, `delete_repo` (cleanup only). Use a dedicated test account or a throw-away token.
- **Headers on every request:** `Accept: application/vnd.github+json`, `X-GitHub-Api-Version: 2022-11-28`.
- **Test data:** every test run creates its own repo with a unique name (`api-test-<timestamp>`), and deletes it in `@AfterClass`. Never rely on pre-existing state.
- **Test case IDs:** one letter prefix per area + 3 digits. A = Authentication, L = Rate limit, U = Users, R = Repositories, I = Issues, C = Comments, B = Labels, F = Contents (files), H = Branches, S = Search, G = Gists, X = Cross-cutting.
- **Priority:** P0 = smoke / must pass, P1 = core regression, P2 = edge / nice to have.
- **Type:** `+` positive, `-` negative, `S` schema, `H` header/contract.
- **Rate limit:** authenticated = 5000 req/h, unauthenticated = 60 req/h. Search is limited to 30 req/min. Keep the suite small enough not to exhaust it.

Suggested suites (TestNG groups): `smoke`, `regression`, `negative`, `contract`.

1. Authentication & Rate Limit

US-01 – As an API consumer, I want to authenticate with a token so that I can access my private resources.

Acceptance criteria

- A valid token returns the authenticated user profile.
- An invalid or missing token is rejected with a clear error.

ID	Title	Request	Expected	Pri	Type
A001	Get authenticated user with valid token	`GET /user`	200, `login` equals the token owner, `id` is a number	**P0**	+
A002	Request without token	`GET /user` (no auth header)	401, `message` = "Requires authentication"	**P0**	-
A003	Request with invalid token	`GET /user`, `Bearer invalid`	401, `message` = "Bad credentials"	**P0**	-
A004	Public resource works without token	`GET /users/octocat`	200	**P1**	+
A005	Response body matches user schema	`GET /user`	Validates against `user-schema.json`	**P1**	S

US-02 – As an API consumer, I want to see my rate limit so that I can avoid being throttled.

ID	Title	Request	Expected	Pri	Type
L001	Rate limit endpoint returns quota	`GET /rate_limit`	200, `resources.core.limit` = 5000 (authenticated), `remaining` ≤ `limit`	**P1**	+
L002	Rate limit headers present on normal responses	`GET /user`	Headers `X-RateLimit-Limit`, `X-RateLimit-Remaining`, `X-RateLimit-Reset` exist	**P1**	H
L003	`Remaining` decreases between calls	two `GET /user` calls	second `X-RateLimit-Remaining` < first	**P2**	H

2. Users

US-03 – As an API consumer, I want to look up public user profiles so that I can display user information.

ID	Title	Request	Expected	Pri	Type
U001	Get existing user	`GET /users/octocat`	200, `login` = "octocat", `type` = "User"	P0	+
U002	Get non-existent user	`GET /users/{random-uuid}`	404, `message` = "Not Found"	P0	-
U003	List public repos of a user	`GET /users/octocat/repos`	200, array, every item has `owner.login` = "octocat"	P1	+
U004	Pagination with `per_page`	`GET /users/octocat/repos?per_page=2`	200, array size ≤ 2	P1	+
U005	Pagination `page` returns different data	`?per_page=1&page=1` vs `page=2`	the two items have different `id`	P2	+
U006	`Link` header exposes pagination	`?per_page=1`	`Link` header contains `rel="next"`	P2	H
U007	Conditional request with ETag	`GET /users/octocat`, then repeat with `If-None-Match: <etag>`	second response 304, empty body	P2	H

3. Repositories

US-04 – As a developer, I want to create a repository through the API so that I can automate project setup.

Acceptance criteria

- A repo is created with the requested name, visibility and description.
- Invalid or duplicate input is rejected with 422.

ID	Title	Request	Expected	Pri	Type
R001	Create repo with minimal body	`POST /user/repos {name}`	201, `name` matches, `private` = false, `owner.login` = me	P0	+
R002	Create private repo with description	`{name, private:true, description}`	201, `private` = true, `description` matches	P1	+
R003	Create repo with `auto_init`	`{name, auto_init:true}`	201; `GET .../contents/README.md` → 200	P1	+
R004	Create repo without name	`{}`	422, `errors[0].field` = "name"	P0	-
R005	Create duplicate repo	same `name` twice	second call 422, message contains "name already exists"	P0	-
R006	Create repo without auth	no token	401	P1	-
R007	Create repo with invalid characters in name	`{name:"bad name!"}`	201 with sanitized name (spaces become `-`) OR 422; assert the actual documented behaviour	P2	-
R008	Response matches repo schema	R001 response	Validates against `repo-schema.json`	P1	S

US-05 – As a developer, I want to read repository details so that I can verify its configuration.

ID	Title	Request	Expected	Pri	Type
R009	Get repo by owner/name	`GET /repos/{owner}/{repo}`	200, `full_name` = "owner/repo"	**P0**	+
R010	Get non-existent repo	`GET /repos/{owner}/{random}`	404	**P0**	-
R011	Get private repo without auth	no token, private repo	404 (GitHub hides private repos)	**P1**	-
R012	List my repos contains created repo	`GET /user/repos`	200, array contains the repo name	**P1**	+
R013	Filter my repos by visibility	`GET /user/repos?visibility=private`	200, all items `private` = true	**P2**	+

US-06 – As a developer, I want to update and delete repositories so that I can manage their lifecycle.

ID	Title	Request	Expected	Pri	Type
R014	Update description	`PATCH /repos/{o}/{r} {description}`	200, new description returned; follow-up GET confirms	**P0**	+
R015	Rename repo	`PATCH {name:new}`	200, `name` = new; old name now 404 (or redirect 301)	**P1**	+
R016	Archive repo	`PATCH {archived:true}`	200, `archived` = true	**P2**	+
R017	Update repo owned by someone else	`PATCH /repos/octocat/Hello-World`	403 or 404	**P1**	-
R018	Delete repo	`DELETE /repos/{o}/{r}`	204; follow-up GET → 404	**P0**	+
R019	Delete already deleted repo	`DELETE` again	404	**P1**	-

4. Issues

US-07 – As a team member, I want to create and manage issues so that work can be tracked.

Precondition: a test repo exists (created in `@BeforeClass`). Issues must be enabled (default).

ID	Title	Request	Expected	Pri	Type
I001	Create issue	`POST /repos/{o}/{r}/issues {title, body}`	201, `number` = 1, `state` = "open", `title` matches	**P0**	+
I002	Create issue without title	`{body}`	422, `errors[0].field` = "title"	**P0**	-
I003	Create issue with labels and assignee	`{title, labels:["bug"], assignees:[me]}`	201, `labels[0].name` = "bug", `assignee.login` = me	**P1**	+
I004	Get issue by number	`GET /repos/{o}/{r}/issues/{n}`	200, same data as created	**P0**	+
I005	Get non-existent issue	`.../issues/999999`	404	**P1**	-
I006	Update title and body	`PATCH .../issues/{n}`	200, fields updated	**P1**	+
I007	Close issue	`PATCH {state:"closed", state_reason:"completed"}`	200, `state` = "closed", `closed_at` not null	**P0**	+
I008	Reopen issue	`PATCH {state:"open"}`	200, `state` = "open", `closed_at` null	**P1**	+
I009	List open issues	`GET .../issues?state=open`	200, all `state` = "open"	**P1**	+
I010	List closed issues	`GET .../issues?state=closed`	200, contains the closed issue only	**P1**	+
I011	Filter by label	`GET .../issues?labels=bug`	200, every item has the label	**P2**	+

ID	Title	Request	Expected	Pri	Type
I012	Issue list excludes nothing created by others in a fresh repo	`GET .../issues?state=all`	size equals number of created issues	P2	+
I013	Create issue without auth	no token	401 (or 404 on private repo)	P1	-
I014	Issue number increments	create two issues	second `number` = first + 1	P2	+
I015	Response matches issue schema	I001 response	Validates against `issue-schema.json`	P1	S

US-08 – As a team member, I want to comment on issues so that I can discuss work.

ID	Title	Request	Expected	Pri	Type
C001	Add comment	`POST .../issues/{n}/comments {body}`	201, `body` matches, `user.login` = me	P0	+
C002	Add comment with empty body	`{body:""}`	422	P0	-
C003	List comments	`GET .../issues/{n}/comments`	200, contains created comment	P1	+
C004	Update comment	`PATCH /repos/{o}/{r}/issues/comments/{id}`	200, `body` updated	P1	+
C005	Delete comment	`DELETE .../issues/comments/{id}`	204; GET → 404	P1	+
C006	Comment count updates on issue	after C001	`GET issue` → `comments` = 1	P2	+
C007	Comment on non-existent issue	`.../issues/999999/comments`	404	P1	-

US-09 – As a maintainer, I want to manage labels so that issues can be categorised.

ID	Title	Request	Expected	Pri	Type
B001	Create label	`POST /repos/{o}/{r}/labels {name, color:"ff0000"}`	201, `name` and `color` match	P1	+
B002	Duplicate label	same name twice	422, `errors[0].code` = "already_exists"	P1	-
B003	Invalid color	`{color:"zzzzzz"}`	422	P2	-
B004	List labels	`GET .../labels`	200, contains created label	P1	+
B005	Delete label	`DELETE .../labels/{name}`	204	P1	+
B006	Update label	`PATCH .../labels/{name} {color, description}`	200, values updated; follow-up GET confirms	P2	+

5. Repository content, branches & commits

US-10 – As a developer, I want to manage files in a repo so that I can automate content updates.

Precondition: repo created with `auto_init: true`.

ID	Title	Request	Expected	Pri	Type
F001	Create file	`PUT .../contents/test.txt {message, content(base64)}`	201, `content.path` = "test.txt", `commit.sha` present	P0	+
F002	Read file	`GET .../contents/test.txt`	200, decoded `content` equals the original text	P0	+
F003	Update file with sha	`PUT` with current `sha`	200, new `commit.sha` differs from previous	P1	+

ID	Title	Request	Expected	Pri	Type
F004	Update file without sha	`PUT` on existing file, no `sha`	422, message mentions "sha"	P1	-
F005	Delete file	`DELETE .../contents/test.txt {message, sha}`	200, `commit` present; GET → 404	P1	+
F006	Read non-existent file	`GET .../contents/nope.txt`	404	P1	-
F007	List directory	`GET .../contents`	200, array with `README.md`	P2	+
F008	Non-base64 content	`{content:"%%%"}`	422	P2	-

US-11 – As a developer, I want to inspect branches and commits so that I can verify history.

ID	Title	Request	Expected	Pri	Type
H001	List branches	`GET .../branches`	200, contains `main`	P1	+
H002	Get single branch	`GET .../branches/main`	200, `name` = "main", `commit.sha` present	P1	+
H003	Get non-existent branch	`GET .../branches/nope`	404	P1	-
H004	List commits	`GET .../commits`	200, array not empty; latest `sha` equals F001 commit sha	P1	+
H005	Create branch via git ref	`POST .../git/refs {ref:"refs/heads/feature", sha}`	201, `ref` = "refs/heads/feature"	P1	+
H006	Create duplicate ref	same call twice	422, "Reference already exists"	P2	-
H007	Delete branch	`DELETE .../git/refs/heads/feature`	204	P2	+

6. Search

US-12 – As an API consumer, I want to search repositories and users so that I can discover content.

ID	Title	Request	Expected	Pri	Type
S001	Search repositories by keyword	`GET /search/repositories?q=rest-assured`	200, `total_count` > 0, `items` not empty	P0	+
S002	Search with language qualifier	`?q=rest-assured+language:java`	200, every `language` = "Java"	P1	+
S003	Sort by stars descending	`?q=rest-assured&sort=stars&order=desc`	200, `stargazers_count` is non-increasing	P1	+
S004	Missing `q`	`GET /search/repositories`	422	P1	-
S005	No results	`?q=<random-uuid>`	200, `total_count` = 0, `items` empty	P1	+
S006	Search users	`GET /search/users?q=octocat`	200, `items[*].login` contains "octocat"	P2	+
S007	Pagination	`?q=java&per_page=5`	200, `items` size = 5	P2	+

7. Gists

US-13 – As a developer, I want to create and delete gists so that I can share snippets.

ID	Title	Request	Expected	Pri	Type
G001	Create gist	`POST /gists {files:{"a.txt":{content:"x"}}, public:false}`	201, `files` contains "a.txt", `public` = false	**P1**	+
G002	Create gist without files	`{}`	422	**P1**	-
G003	Get gist	`GET /gists/{id}`	200, content matches	**P1**	+
G004	Update gist	`PATCH /gists/{id}`	200, content updated	**P2**	+
G005	Delete gist	`DELETE /gists/{id}`	204; GET → 404	**P1**	+
G006	Create gist without auth	no token	401	**P2**	-

8. Cross-cutting (contract) tests

ID	Title	Check	Pri	Type
X001	Content type	Every JSON response has `Content-Type: application/json; charset=utf-8`	**P1**	H
X002	Response time	Core GETs respond in under 2 s	**P2**	H
X003	Error body shape	Every 4xx response has `message` and `documentation_url`	**P1**	H
X004	Unsupported method	`DELETE /user` → 404 or 405	**P2**	-
X005	Wrong API version header	`X-GitHub-Api-Version: 1999-01-01` → 400	**P2**	-
X006	Unknown route	`GET /this-does-not-exist` → 404	**P2**	-

9. Suggested execution order & coverage

Suite	Contents	Test cases
smoke (P0)	A001-A003, U001-U002, R001, R004, R005, R009, R010, R014, R018, I001, I002, I004, I007, C001, C002, F001, F002, S001	21
regression (P0 + P1)	all of sections 1–7	89
negative	every test of type `-`	29
contract	every `S` and `H` test, section 9	10

Total: 96 test cases across 13 user stories.

11. Recommended Automation Approach

- `RequestSpecification` / `ResponseSpecification` in a base class instead of repeating headers.
- POJOs for request bodies (Jackson) instead of raw `Map`s, and POJO deserialization in assertions.
- JSON schema validation with `rest-assured:json-schema-validator`.
- `@DataProvider` for the negative cases (invalid token, empty title, invalid color, ...).
- TestNG reports (Surefire) for the HTML report, and a GitHub Actions workflow that runs the suite with `GITHUB_TOKEN` as a repo secret.